《LLM-based Agents Suffer from Hallucinations: A Survey of Taxonomy, Methods, and Directions》精读笔记

> 原文：Xixun Lin 等（中国科学院信息工程研究所等），arXiv:2509.18970v2, 2025-11。 本笔记为中文学习笔记，以要点归纳和个人解读为主，不是原文译文；形式化定义、完整文献映射与附录示例请以原文为准。

一句话结论

Agent 的幻觉不只是"说错话"，而是在推理、执行、感知、记忆、通信任何一个环节都可能出现的虚构或误判的"类人行为"。作者用 POMDP 建模 Agent 循环，提出"内部状态（信念） + 外部行为"的划分，归纳出 5 大类幻觉、18 个触发原因，总结了 10 类缓解方法和对应的检测手段，并给出 6 个未来方向。

1. 为什么 Agent 幻觉更棘手

和单个 LLM 的幻觉相比，Agent 幻觉有三个特点：

1. 类型更多样：它是多个模块耦合产生的复合行为，而不只是一个回答错了。
2. 传播链更长：跨多步、多次状态转移，中间环节（感知、推理）的错误会一路累积。
3. 后果更严重：会产生"物理层面的后果"，错误动作可能直接影响任务、设备和用户。

2. 形式化：POMDP 与 Agent 循环

- 交互环境用 8 元组描述：状态、动作、转移、目标、观察、观察函数、奖励、折扣因子。关键在于"部分可观测"：Agent 看不到真实状态，只能维护一个信念状态 b_t。
- 单 Agent 每轮循环：
    1. 推理：根据信念和目标生成计划 p_t
    2. 执行：把计划变成动作 a_t（通常是调用工具）
    3. 反馈：环境给出奖励 r_t
    4. 环境转移：进入下一个状态
    5. 感知：生成观察 o_{t+1}
    6. 记忆：用动作和观察更新记忆 m_{t+1}
    7. 信念更新：综合以上信息更新 b_{t+1}
- 多 Agent 系统（MAS）额外多两步：广播（按计划向邻居发送消息）和结构演化（通信拓扑随时间变化）。
- Agent 幻觉的定义要点：在推理、执行、感知、记忆、通信中，由过度自信的"类人行为"导致的虚构或错误操作。它往往是 Agent 自身认知过程内生的，并且会跨模块传播、不易察觉。

3. 分类体系：5 大类幻觉与 18 个触发原因

3.1 推理幻觉

推理分三个阶段：目标理解 → 意图分解 → 计划生成，对应三种子类型：GUH（目标理解幻觉）、IDH（意图分解幻觉）、PGH（计划生成幻觉）。

触发原因	要点
目标表达有问题	目标描述不完整或含歧义；对策是让 Agent 主动澄清
主观理解不足	指令遵循出现偏差；长上下文里只盯着最近的 token
依赖建模不足	子意图遗漏、冗余或顺序错乱
规划信息误解	误解可操作对象之间的关系（空间、时间、社会关系）；超出自身知识边界时过度自信
规划信息误用	逻辑谬误（规则冲突、因果倒置）；内省能力不足，误读反馈或误以为推理已经自洽

3.2 执行幻觉

执行分为工具选择（先从全部工具中检索候选，再选一个）和工具调用（填参数）。对应两种子类型：TSH（选了不存在或不相关的工具）和 TCH（参数错误、遗漏或凭空编造）。其共同特征是：声称做完了某一步，实际上并没有完成。

触发原因	要点
工具文档有缺陷	冗余、不完整或不准确、缺乏统一规范
对工具模式理解浅	同一工具可用于多种场景、工具之间有顺序依赖或嵌套调用，而训练时见得太少
动态适应能力弱	API 已变更或废弃，模型参数里的知识还停在过去
缺乏"能否解决"的意识	没有合适工具时硬选一个；计划不清楚时编造参数

3.3 感知幻觉

把外部状态转成内部观察时，与真实环境明显不符。

触发原因	要点
环境传感器故障	镜头畸变、惯性测量单元漂移等
编码能力有限	单一模态表示不充分；跨模态对齐和融合弱；生成时被语言先验主导，忽略了非文本信息

3.4 记忆幻觉

Agent 默认自己的记忆是可靠的，不加校验就拿来用。分为 MRH（检索到不相关或不存在的内容）和 MUH（错误地修改或删除记忆）。

触发原因	要点
初始记忆有缺陷	带有偏见或信息缺失，Agent 用假设去"脑补"
检索机制欠佳	排序只看表面相似；索引粒度不当或更新滞后；误解查询意图
优先级分配不当	遗忘时删掉了重要内容；合并记忆片段时引入冲突（包括隐性冲突）

写入信息不准确	摘要压缩丢失细节或产生扭曲；存储格式不统一；容量有限只能选择性保留

3.5 通信幻觉（多 Agent 系统）

表面上在有效沟通，实际交换的信息不准确或是编造的。

触发原因	要点
错误消息传播	发出的消息本身有事实性或忠实性问题；内容冗余淹没关键信号；角色不同导致信息不对称
通信协议不协调	异步调度造成信息丢失或过载；自然语言格式有歧义（建议改用 JSON 等结构化格式）；缺少容错设计
网络更新无效	拓扑更新滞后，消息被发给不合适的对象

4. 缓解方法：3 个方向、10 类方法

4.1 知识利用

- 外部知识引导（EKG）
  - 专家知识：三种形式，即知识库（反思、动作、经验类）、规则（数学规则、语言规则、流程规则、MAS 通信协议）、数据构造（指令对、问答对、偏好对）。
  - 世界模型：物理规律、社会规范等常识。例如 NavMorph 用世界模型补充对物理环境的认知，WKM 补充"做饭前要先洗菜"这类常识。
- 内部知识增强（IKE）
  - 知识激活：CoT、ToT、约束式提示。
  - 知识修正：知识编辑（先定位再编辑，或用元学习）；知识遗忘（参数优化或参数合并）。

4.2 范式改进

方法	作用机制
对比学习（CoL）	通过比较样本异同学到更有区分度的表示
课程学习（CuL）	由易到难，先积累成功经验
强化学习（RL）	试错、反馈、调整；另有工作致力于提高奖励信号的可靠性
因果学习（CaL）	建模因果关系而不只是相关性，缓解感知和推理幻觉
图学习（GL）	工具图（如 ControlLLM）；学习 MAS 的通信拓扑
解码优化（DO）	推理时生效：对比解码、自校准注意力、融合解码（如 CGD）

4.3 事后验证

每完成一步就校验，防止错误累积。

- 自我验证（SM）：自我反思（可以结合置信度估计）、自一致性（多条路径投票）、自我提

问。
- 外部验证器（VA）：五类，分别基于语言（拆原子事实做蕴含判断）、检索（对照搜索结果）、执行（运行代码或计划）、仿真（在沙盒中验证）、集成（组合多种验证器）。

原文表 I 给出了"幻觉类型 × 方法"的覆盖矩阵。粗略来看：推理和执行类幻觉几乎各种方法都有研究；记忆检索和通信类幻觉覆盖较少，很多格子还是空白。

5. 检测方法

- 原文图 4 按幻觉类型列出了代表性检测工作，例如：工具选择方面有 ToolBH、When2Call、Gorilla、RelyToolBench；通信方面有 MAST、Who&When、AgentAuditor 等。
- 观察：感知幻觉的检测工作最多，记忆和通信幻觉的检测最少。作者的解释是，感知处在 Agent 的"浅层"，错误容易定位；记忆和通信处在"深层"，最终输出与大量中间模块耦合，难以追溯。

6. 六个未来方向

1. 幻觉累积研究：从单轮分析转向多步决策全过程，尽早发现和阻断。
2. 准确定位幻觉来源：建模并追踪完整执行轨迹，例如在每个阶段插入轻量检查点。
3. 机理可解释性：把特征分析、因果路径追踪等方法扩展到多步交互的 Agent。
4. 统一基准：现有基准大多只覆盖一类幻觉（如 HallusionBench、MemoryAgentBench、ToolBH），缺少覆盖全部五类的综合基准。
5. 持续自我进化：用终身学习应对不断变化的目标和环境。
6. 基础架构升级：线性复杂度模块、神经符号系统、自动化架构搜索；以及能自主调度的动态 Agent 系统，替代固定工作流。

7. 对本仓库 web-search-agent-go 的启示（个人解读）

- 用这套分类体系对照本仓库：
    - 执行幻觉：模型声称搜过了但实际上没有调用工具。可以让验收检查"答案里提到的来源是否真的出现在工具调用记录里"。
    - 推理幻觉里的子意图遗漏：多步问题只搜了一部分就作答。
    - 缺乏"能否解决"的意识：没有搜索能力时，本仓库在 `config.Resolve` 中默认拒绝回退到无搜索后端（需要显式传 `-allow-no-search`），这正是在防范这类幻觉。
- 方向 2（轨迹追踪、检查点）与本仓库 `internal/evidence` 的轨迹落盘和凭证扫描思路一致，可以在此基础上给每一步加结构化的校验结果。
- 外部验证器里的"检索型验证器"和"执行型验证器"，都可以作为验收层的增强手段。